CPSC 2150 Project Report

Yash Patel

Requirements Analysis

Functional Requirements:

1. As a player, I need to place a marker so that I can indicate my turn
2. As a player, I want to restart the game after drawing so I can continue playing.
3. As a player, I need to alternate players so that I can have each of them have a turn playing.
4. As a programmer, I need to have a computer to play with, so I can view and test my game.
5. As a player, I need to be able to place my marker anywhere on a 5 x 8 grid, so then I can use my turn.
6. As a gamescreen, I need to know when all the spaces are filled with no winning side so that it can display the draw message.
7. As a player, I want to be able to quit the game so that I can stop playing
8. As a player, I need to be player 'X' if I am first so that I can start the game
9. As a gameboard, I need to place either 'X' or 'O', so I know the game pieces
10. As a gameboard, I need to check to see if the row/col is valid so that I can place a marker there
11. As a gameboard, I need to know if 5 of the same markers are placed in the same column to count as a vertical win
12. As a gameboard, I need to know if 5 of the same markers are placed in the same row to count as a horizontal win
13. As a gameboard, I need to know if 5 of the same markers are placed in a diagonal line to count as a diagonal win

Non-Functional Requirements

1. The program is written in java
2. The program is played using the terminal
3. The program must have two players
4. The program must be developed in IntelliJ
5. The player must know the rules of tic-tac-toe

6. the program must have a 5 x 8 board
7. The ‘X’ Player must go first

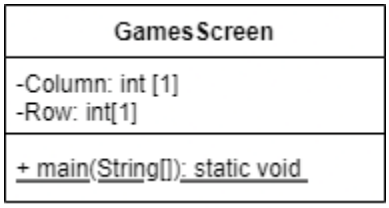
Deployment Instructions

Details in Projects 2-5.

System Design

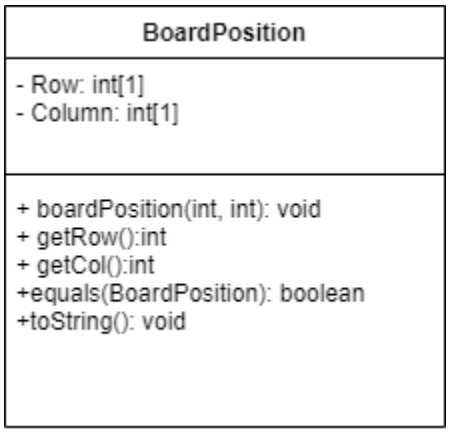
Class 1: GameScreen

Class diagram



Class 2: BoardPosition

Class diagram



Class 3: GameBoard

Class diagram

GameBoard
- PlayerChar: char[*] {'X', 'O'} - Board: char[*][*]
+ GameBoard(): void + checkSpace(BoardPosition): boolean + placeMarker(BoardPosition, char): void + checkForWinner(BoardPosition): boolean + checkForDraw(): boolean + checkHorizontalWin(BoardPosition, char): boolean + checkVerticalWin(BoardPosition, char): boolean + checkDiagonalWin(BoardPosition, char): boolean + whatsAtPos(BoardPosition): char + isPlayerAtPos(BoardPosition, char): boolean + toString():string

Class 4: IGameBoard

Class diagram

IGameBoard
-rowNum: int[1] -colNum: int[1] -winCount: int[1]
+ getNumRows(): int <default> + getNumColumns(): int <default> + getNumToWin(): int <default> + checkSpace(BoardPosition): boolean <default> + placeMarker(BoardPosition, char): void + checkForWinner(BoardPosition): boolean <default> + checkVerticalWin(BoardPosition): boolean <default> + checkHorizontalWin(BoardPosition): boolean <default> + checkDiagonalWin(BoardPosition): boolean <default> + checkForDraw(): boolean <default> + whatsAtPos(boardPosition): char + isPlayerAtPos(boardPosition, char): boolean <default>

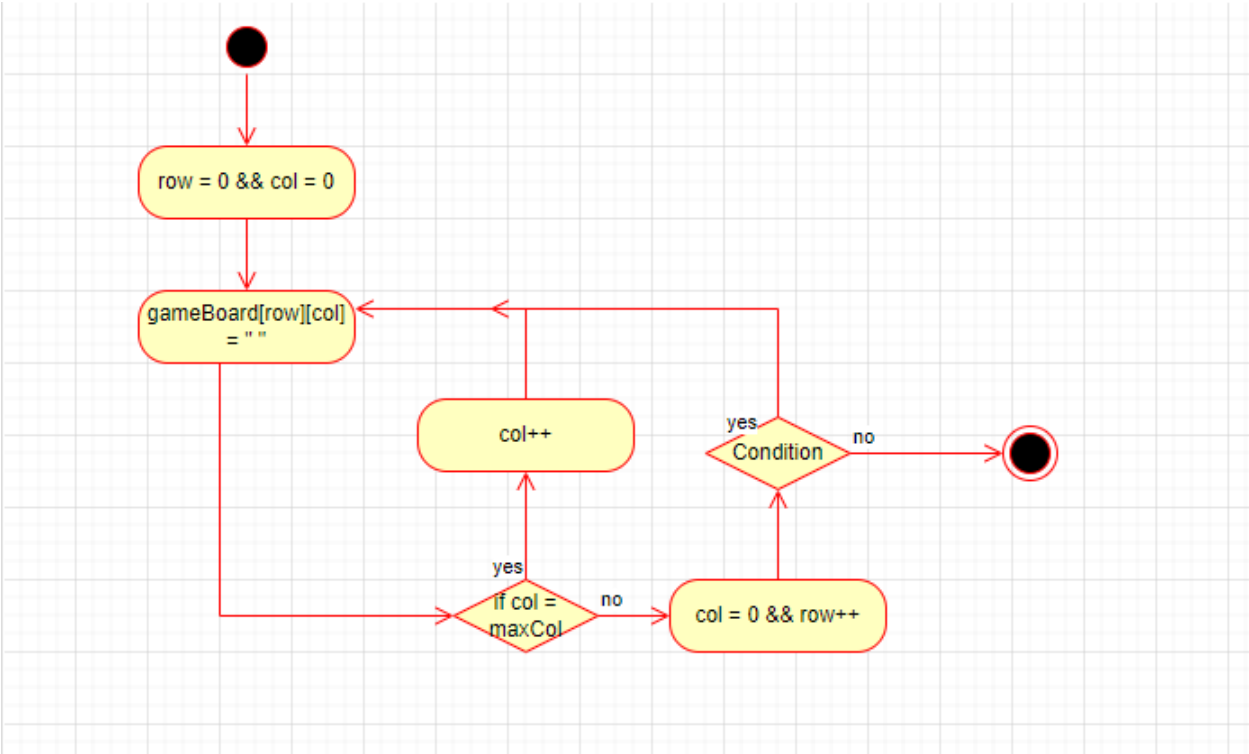
Class 5: AbsGameBoard

Class diagram

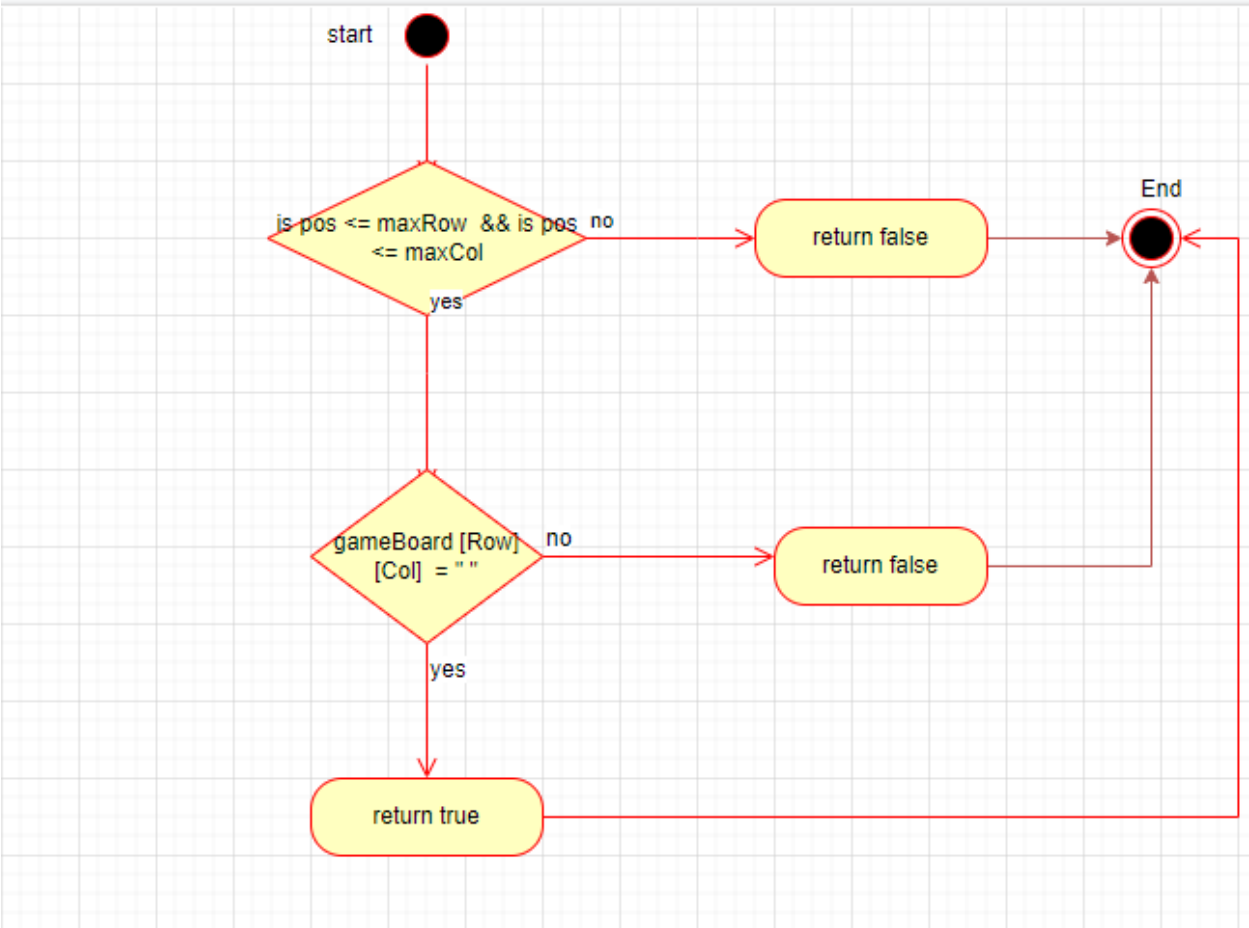
AbsGameBoard
+ toString(): string

Activity diagrams

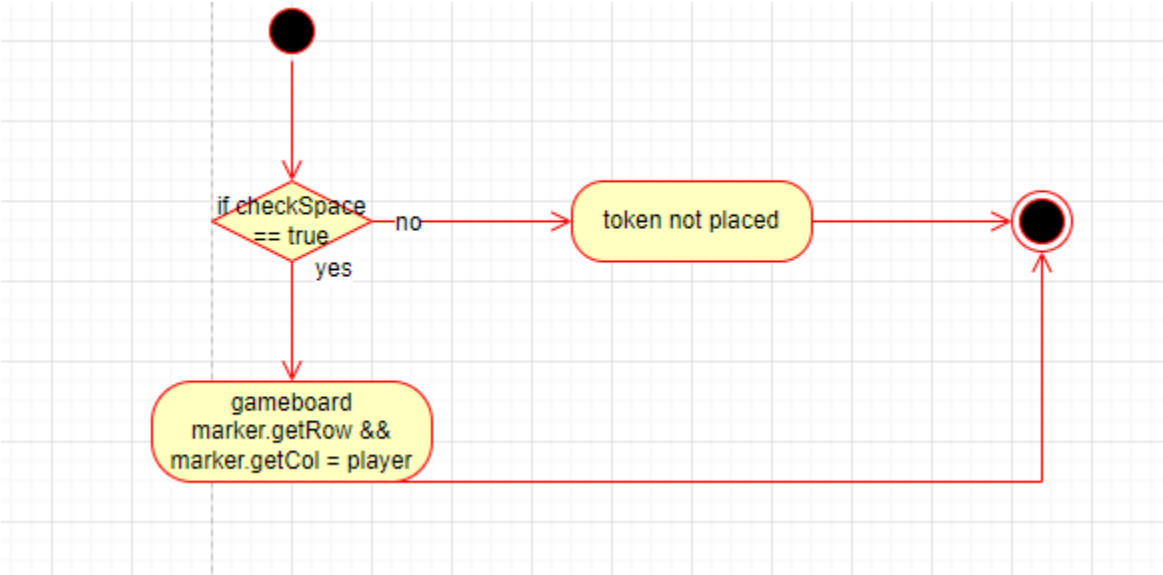
Gameboard(Constructor):



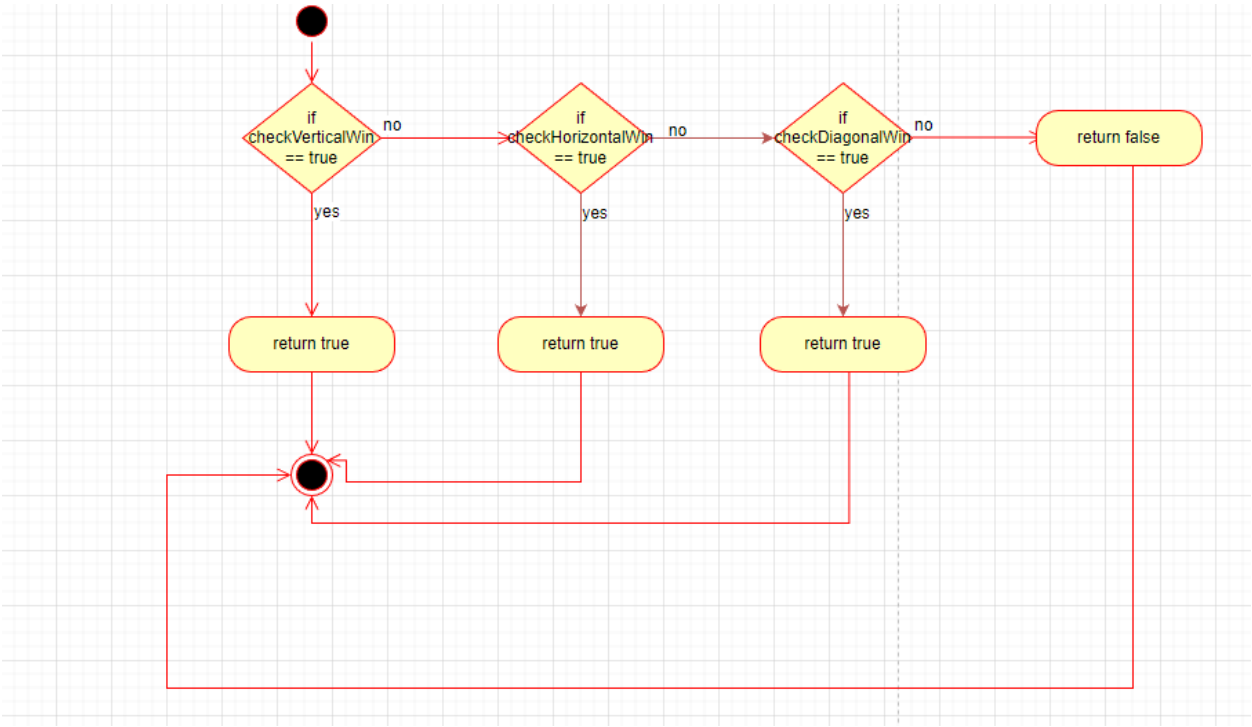
public boolean checkSpace(BoardPosition pos):



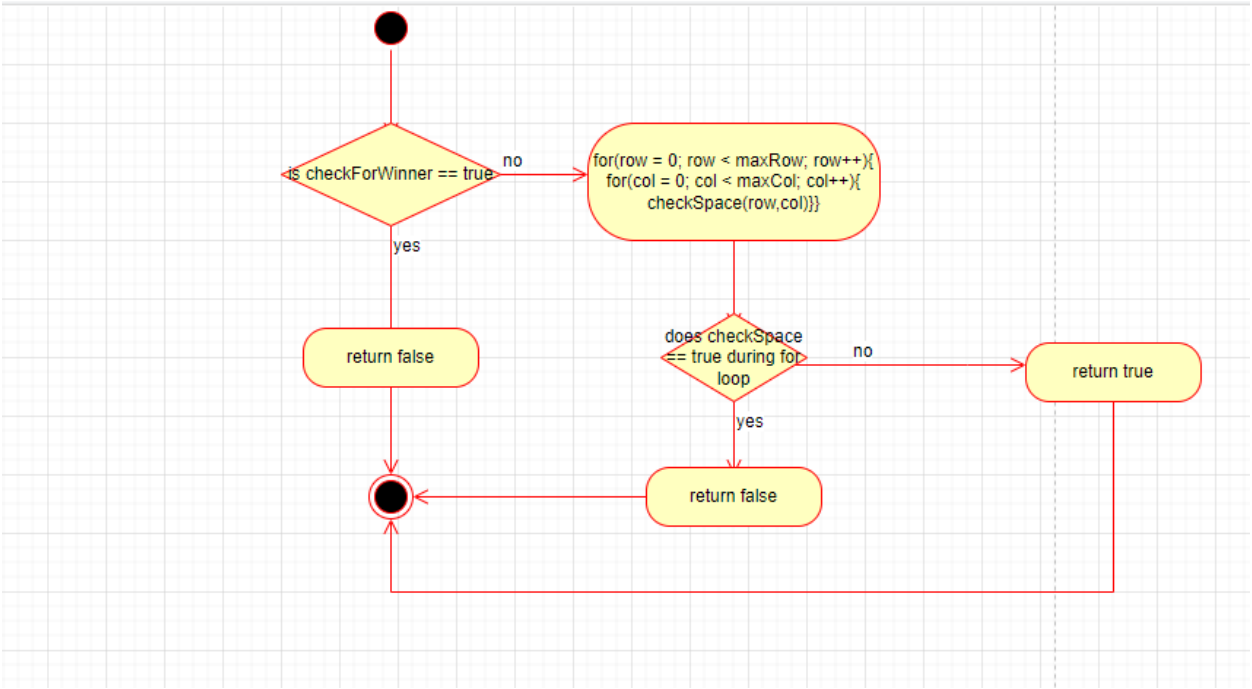
public void placeMarker(BoardPosition marker, char player):



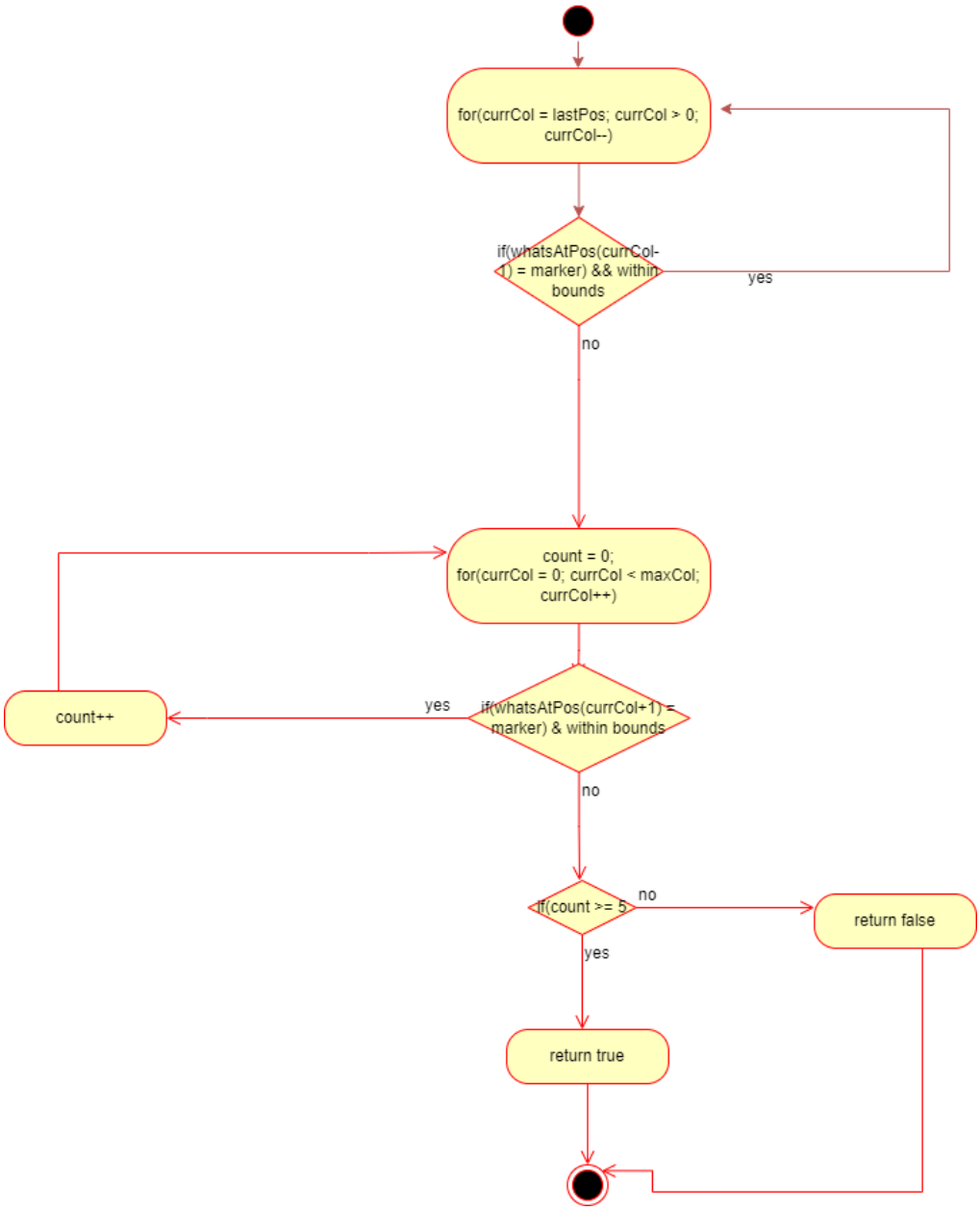
public boolean checkForWinner(BoardPosition lastPos):



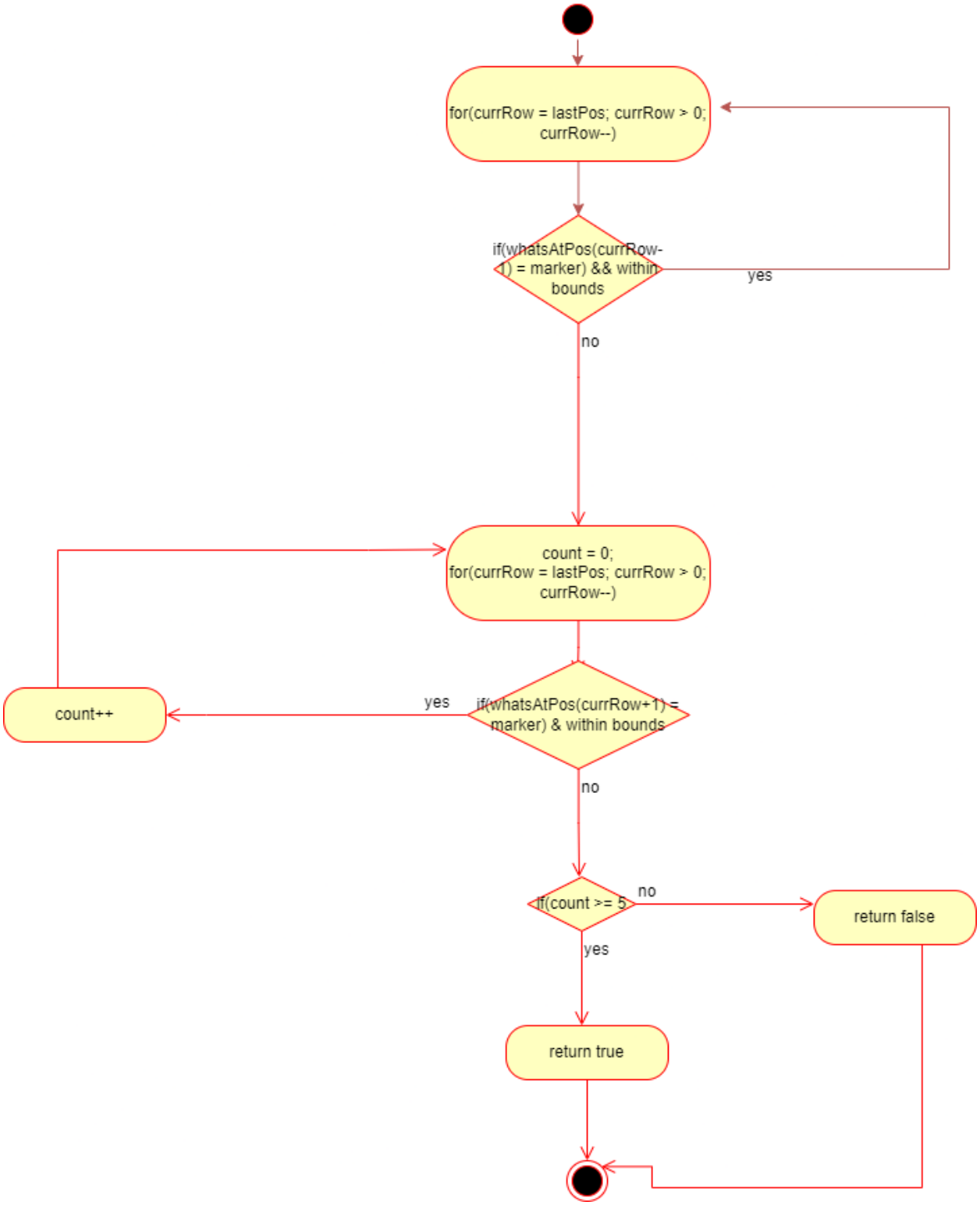
public boolean checkForDraw():



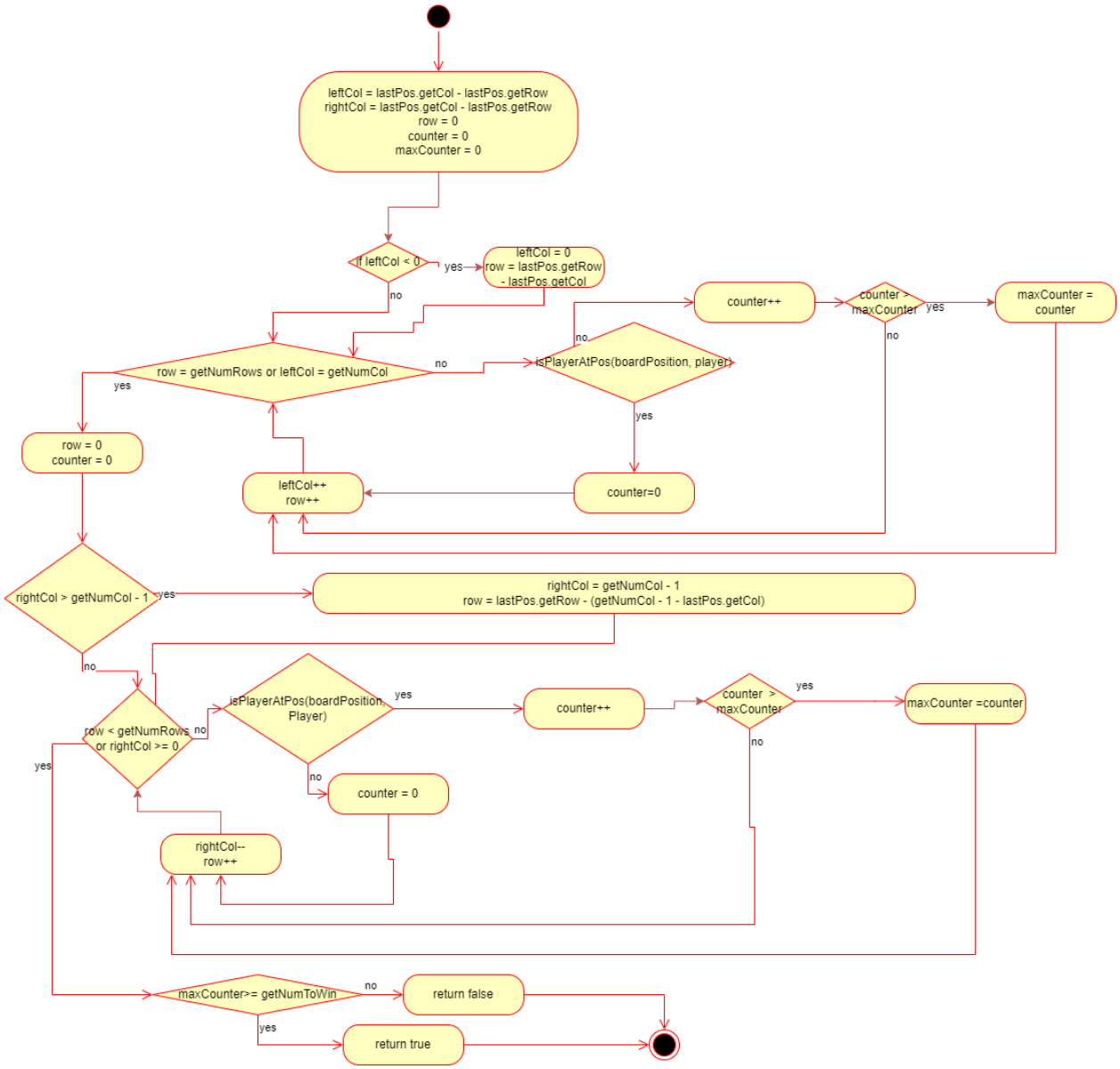
public boolean checkHorizontalWin(BoardPosition lastPos, char player):



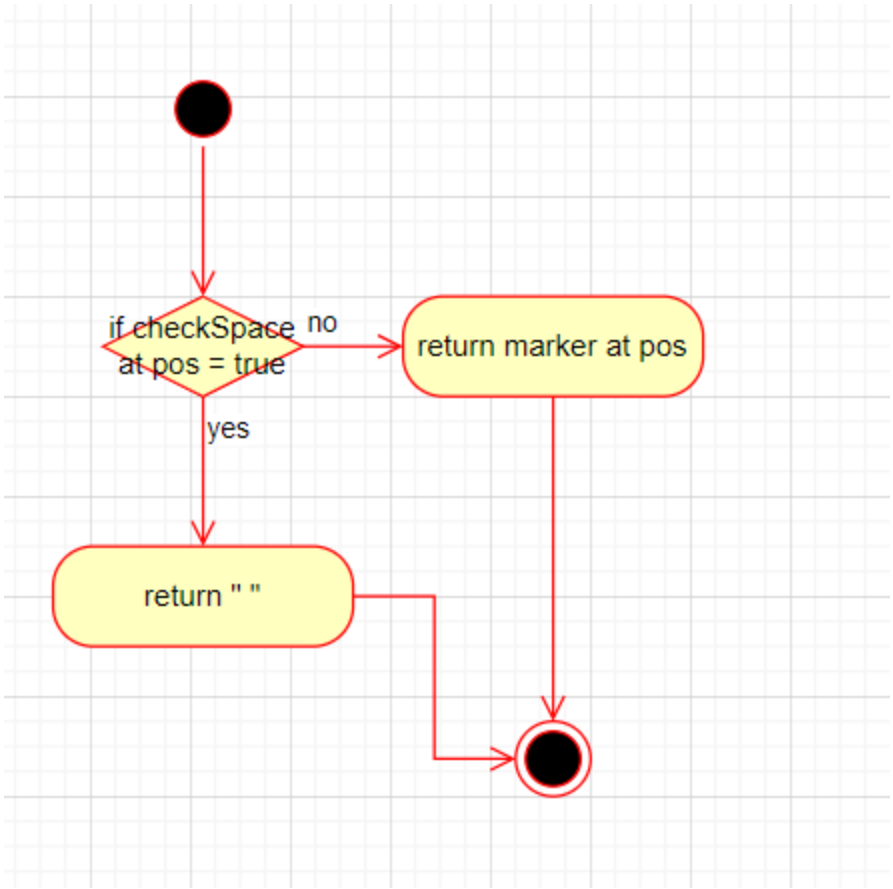
public boolean checkVerticalWin(BoardPosition lastPos, char player):



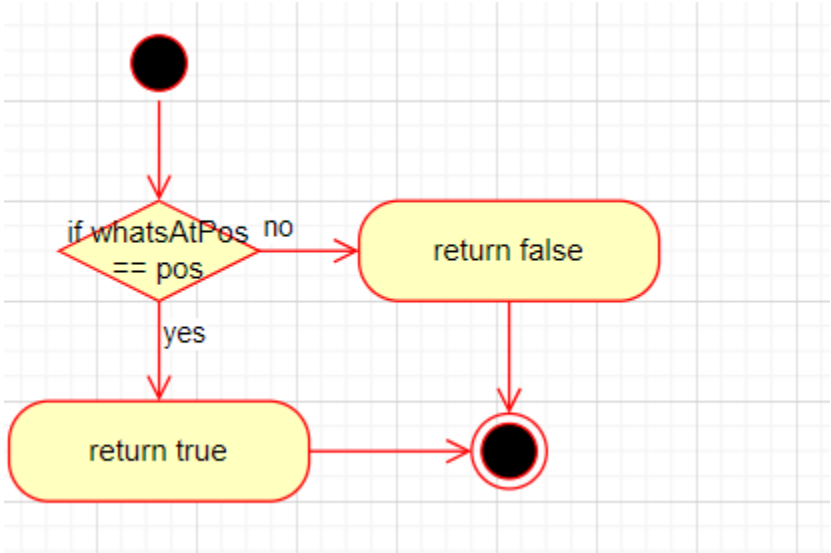
public boolean checkDiagonalWin(BoardPosition lastPos, char player):



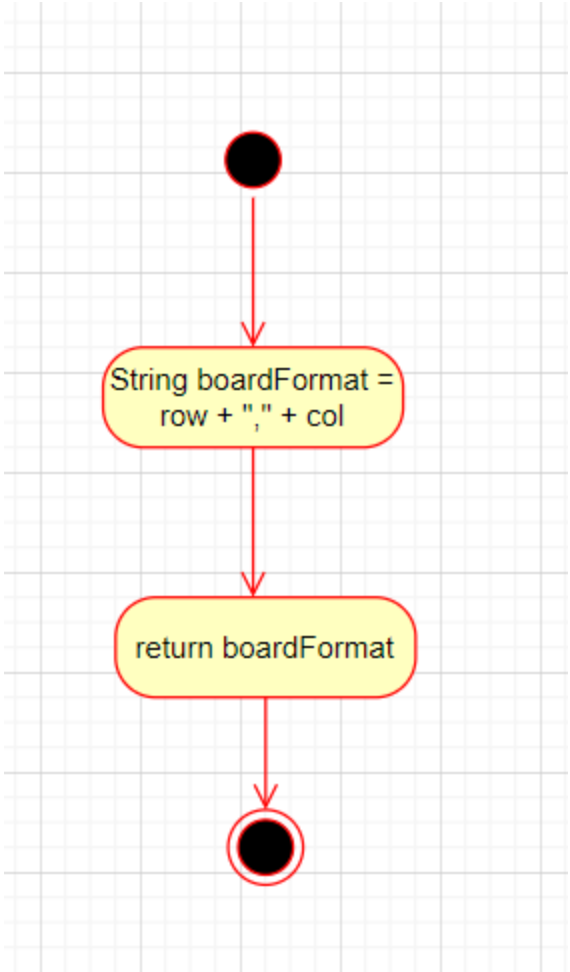
public char whatsAtPos(BoardPosition pos):



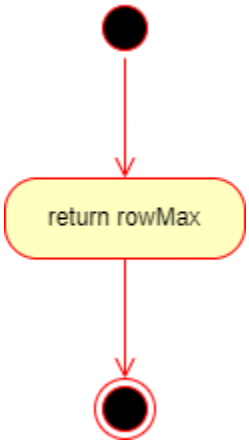
public boolean isPlayerAtPos(BoardPosition pos, char player):



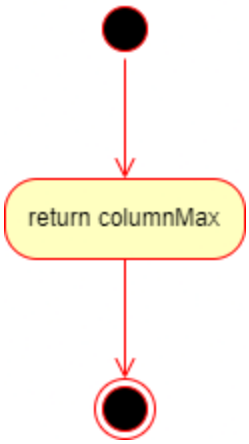
toString():



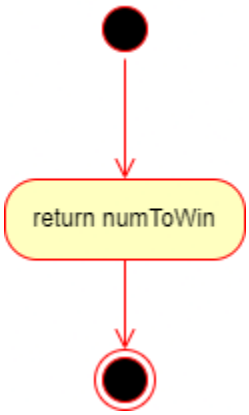
getNumRows():



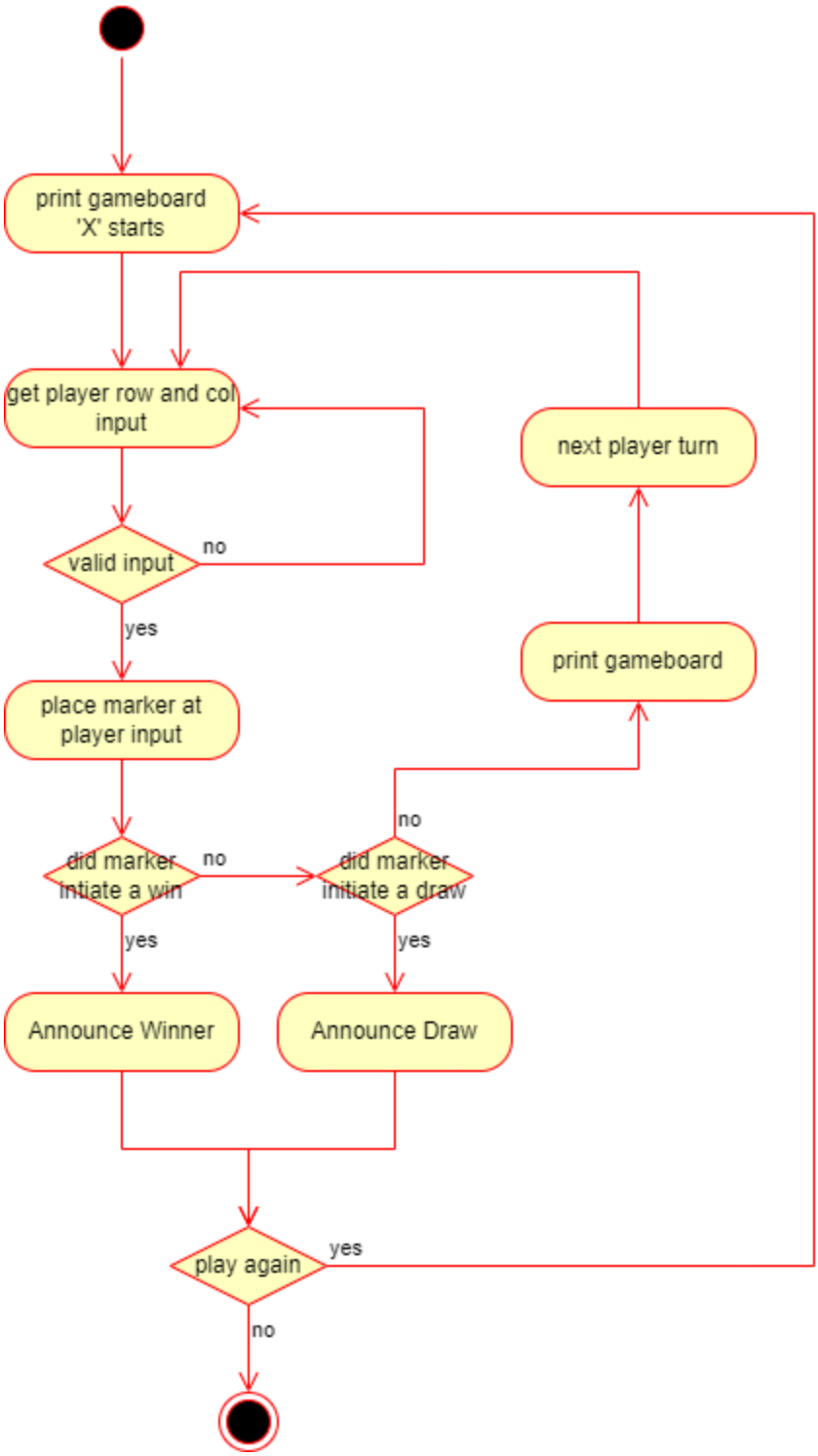
getNumCol();



getNumToWin():



main():



How To Run:

Once inside the Src directory, type “make” to compile code; then “run” to run the code; once the code has successfully finished, type “clean” to removes your compiled (.class) files.

Test Cases

Details in Project 4.